

Decentralized Trade Data Exchange Network

Architecture Blueprint and POC Implementation Snapshot

Status: Architecture proposal with working prototype | **Version:** 0.2 | **Scope:** Indian domestic B2B trade, triple-entry accounting, distributed ledger, privacy-preserving verification

1. Executive summary

The target network is decentralized infrastructure through which registered businesses exchange structured trade documents and record shared accounting events. Its core is a distributed, append-only triple-entry ledger: counterparties sign a common event, retain verifiable records, and derive their respective accounting entries from that event.

The ledger is designed to work with private, encrypted document exchange. Ledger participants should be able to verify event integrity and protocol validity without automatically reading every invoice or attachment. A business can authorize a named third party to verify a defined set of records or claims, with disclosure limited to the permission granted.

Four node classes are proposed: **mother**, **institution**, **professional**, and **light** nodes. These describe target operational responsibilities and capabilities; they do not, by themselves, grant access to plaintext or governance power. The current TypeScript POC implements canonical trade documents, signing and encryption primitives, bilateral ledger proposals, a hosted HTTP sandbox, a Zoho Books adapter, and a company-owned AI gateway. It is a single-process prototype, not a distributed production network or security certification. Consensus, production identity and key custody, retention obligations, and specific external-chain choices remain open.

2. Goals and non-goals

Goals

- Exchange canonical, signed trade documents between registered businesses.
- Create a shared, append-only accounting event for each agreed trade event.
- Let counterparties independently verify signatures, document bindings, event ordering, and ledger integrity.
- Keep contents confidential from nodes and parties not authorized to see them.
- Let a business grant scoped third-party verification or disclosure.
- Support distributed operation across independently governed node operators.
- Support versioned projections for Indian and international trade rails without making any one rail the owner of the canonical model.

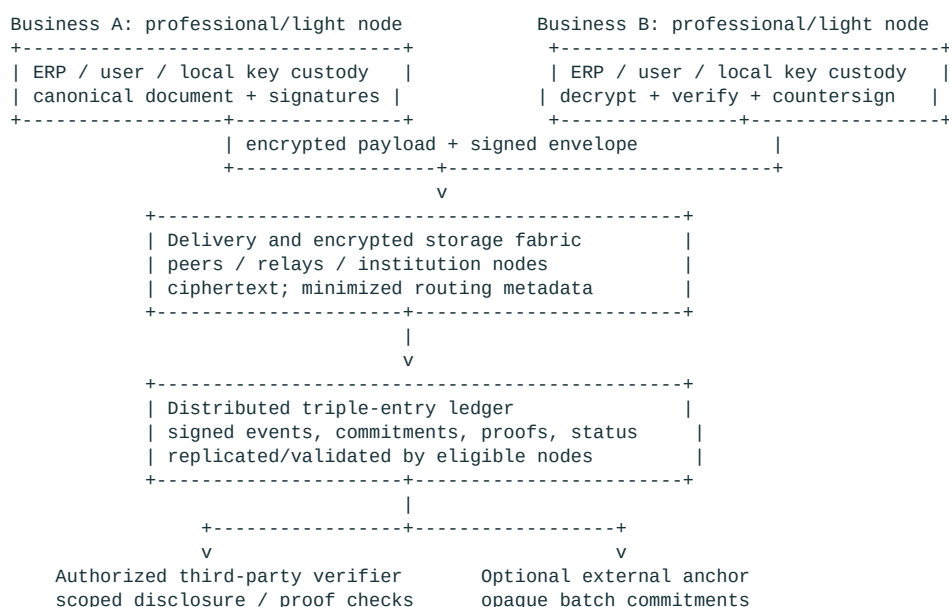
Non-goals for the first release

- Putting plaintext invoices, attachments, tax values, or private keys on a public blockchain.
- Requiring zero-knowledge proofs (ZKPs) for every exchange.
- Treating a blockchain, node operator, or cryptographic proof as proof that a real-world claim is truthful.
- Building every external integration before the core exchange, ledger, and privacy model are proven.

3. Architecture principles

1. **One logical network, distinct data and consensus responsibilities.** Exchange and ledger events share identities, identifiers, and protocol rules. Bulk encrypted payload storage need not be replicated by every validator.
2. **Privacy by default.** Nodes receive the minimum data needed for routing, validation, and availability. Plaintext access is explicit and audience-bound.
3. **Participant-controlled authority.** Businesses control signing keys and grant access. Operating infrastructure does not itself confer authority over trade data.
4. **Append-only accounting.** Corrections append reversing, superseding, or dispute events; they do not rewrite accepted history.
5. **Verifiable claims, scoped disclosure.** A verifier receives only the records, fields, or proofs needed for its purpose.
6. **Explicit trust assumptions.** Validator governance, identity issuance, key recovery, software upgrades, and external services are documented boundaries.

4. Logical architecture



This is the target logical view, not a diagram of the current deployment. A first production implementation may use a limited validator group while defining an open protocol and independently operated nodes. Decentralization is a property of governance and operation, not simply the number of deployed servers.

Current POC deployment boundary

The current runtime is a single Node.js process, bound to loopback by default (port 3000). Its HTTP API supports public-key-only participant registration, health and participant queries, submission of pre-encrypted signed envelopes, the legacy bilateral proposal routes, and a native TEA event lifecycle through `POST /tea/events`, `GET /tea/events`, and `GET /tea/transactions/{id}`. TEA events support a signed SUBMITTED event followed by a counterparty ACCEPTED or DISPUTED event. Except for the minimal health endpoint, routes require a bearer token when one is configured. Binding outside loopback requires both a bearer token of at least 32 bytes and TLS; the executable reads the certificate and key from `TLS_CERT_PATH` and `TLS_KEY_PATH`. Browser origins are denied unless explicitly allowlisted in the runtime configuration. JSON request bodies are size-limited (1 MiB by default), and API responses are not cacheable. Participants, envelopes, event IDs, and ledger records are held in process memory; a restart loses this state. There is no distributed peer discovery, relay fabric, validator consensus, durable storage, or webhook delivery.

The HTTP sandbox is for controlled integration testing only, not a trusted production service. Its public registration route rejects private-key fields; the envelope route accepts ciphertext already produced and signed by the participant, verifies the sender signature and ciphertext hash, and returns only the envelope. Semantic accounting transactions now produce

balanced seller and buyer posting sets for INR invoices and credit notes. Zoho decimal amounts are converted to INR paise, and tax totals must reconcile to explicitly classified GST components; unsupported currencies and ambiguous tax breakdowns are rejected. This protocol issues no native currency and uses no external blockchain. A shared TEA event contains a salted commitment, not the postings themselves. The recipient must decrypt the source document, derive the same transaction, verify the commitment, and only then sign acceptance. The ledger enforces signatures and lifecycle transitions; it cannot determine whether the original invoice or accounting interpretation is truthful. Keep key generation, signing, and encryption in participant-controlled gateways. The in-process helper APIs are intended for local tests and must not be used to load company private keys into a shared hosted process. Production still requires durable storage, business identity proofing and key lifecycle, scoped participant authorization, rate limiting, monitoring, and independent security review.

5. Node classes

Class describes role, service level, and resource profile. A separate capability and authorization model determines what a node may do.

Class	Primary responsibilities	Data access boundary	Must not imply
Mother	Bootstrap, signed network configuration, protocol/software distribution, discovery entry points, network health	Public metadata and ciphertext only when acting as relay	Root authority, plaintext access, unilateral history control
Institution	High-availability relay/storage, possible validator participation, institutional integrations	Ciphertext; only ledger fields allowed by privacy rules	Automatic access to member documents or unilateral event approval
Professional	Full business/service-provider node, ERP connector, document workflow, ledger verification, optional relay	Its operator's authorized plaintext and permitted ledger view	Access to another business's records without a grant
Light	Mobile/browser/constrained participant; submits and verifies relevant events through peers/relays	Its keys and authorized records; limited local history	Blind trust in relay for signatures, inclusion, or ledger verification

Mother-node functions should be replicated across independent operators and governed transparently. Validator membership, bootstrap service, and business-identity issuance are separate roles.

6. Triple-entry accounting and event lifecycle

6.1 Accounting event

Each event is bound to a canonical document revision and carries or commits to the fields needed to derive accounting entries. A conceptual event includes:

- stable event/document identifiers, document kind, revision commitment;
- seller and buyer identities, possibly represented by authorized identifiers or commitments in privacy-sensitive views;
- debit/credit legs and tax components protected according to the chosen visibility model;
- event time, protocol version, predecessor reference;
- submitter signature followed by the counterparty signature required for confirmation;
- optional consent, disclosure, dispute, reversal, or proof references.

Both parties derive their books from the same accepted event. Non-financial events (order, dispatch, goods receipt) may produce commitment/accounting entries without profit-and-loss effects.

6.2 Lifecycle

1. Sender creates and validates a canonical document locally against its profile.

2. Sender commits to the revision, signs the document/envelope, encrypts the payload to the recipient, and appends a provisional submission event.
3. Relays route and optionally replicate ciphertext; they cannot decrypt it by default.
4. Validators check protocol rules, sender authorization, signatures or proofs, uniqueness/idempotency, and allowed state transitions within their permitted view.
5. Recipient decrypts and validates, verifies the transaction commitment, then accepts or disputes. Acceptance produces a signed counterparty event; the sender cannot confirm its own submission.
6. The event reaches defined ledger finality. Both parties verify the event and derive their books.
7. Corrections append new events. Disputes affect derived status without editing the original event.

6.3 Immutability and finality

The target is append-only, tamper-evident history with explicit consensus finality. It is not sound to promise data is impossible to alter under every failure, governance action, or key compromise. The protocol must define validator thresholds, equivocation handling, checkpoints, recovery, and finality.

Encrypted payload retention is distinct from ledger immutability. A commitment may remain verifiable even if payload replicas become unavailable or keys are destroyed. This trade-off between permanent evidence, business retention, and privacy/erasure obligations needs legal review before policy is fixed.

7. Privacy and cryptographic design

7.1 Baseline for first exchange

- End-to-end encryption of documents and attachments to intended recipients.
- Participant-controlled signing keys; authenticated key discovery, rotation, revocation, and documented recovery.
- Signed canonical serialization and versioning to prevent ambiguous hashing/signing.
- Randomized commitments for sensitive or low-entropy values; do not publish a bare hash of predictable invoice data.
- Encryption for stored replicas and backups; no plaintext in logs, analytics, error reports, or default telemetry.
- Replay protection, idempotency, audience binding, and expiry for messages and permissions.

Use established, reviewed libraries and protocols, not custom cryptographic primitives. Final algorithms depend on client support, key storage, interoperability, and independent review.

7.2 Ledger visibility choices

Ledger visibility remains an explicit decision because it determines privacy and validator capabilities:

- **Visible accounting fields:** simplest verification/audit, but exposes counterparties and possibly commercial values to ledger members.
- **Private channels or partitioned state:** limits visibility to authorized groups, but increases operational and governance complexity.
- **Commitments plus proofs:** hides values while allowing selected rule checks; requires well-specified circuits, trusted inputs, and proof lifecycle governance.
- **Encrypted fields plus participant signatures:** protects confidentiality, but validators cannot verify hidden arithmetic without proofs or additional trust assumptions.

These can be combined. Define visibility independently for counterparties, validators, other network participants, and authorized auditors.

7.3 Zero-knowledge proofs

ZKPs are targeted capabilities, not the encryption layer. Possible later uses include proving a document has authorized signatures, a tax computation follows a published rule, a receivable meets lender criteria, or an amount is within a range without revealing its exact value.

For every proof, specify the exact statement, input sources, binding to document revision, circuit/version governance, performance, and handling of bugs or rule changes. A proof validates a computation over supplied inputs; it does not prove those inputs describe reality.

8. Consent-based third-party verification

A business issues a signed, scoped grant binding:

- authorizing business and verifier identity;
- specific documents, events, fields, or claim types;
- purpose, issue time, expiry, and permitted actions;
- onward-disclosure policy;
- verification method and protocol version;
- revocation state or revocation-check method.

Modes range from authenticity/integrity checks, to selected-field disclosure, to proof of a defined claim, to full document access. The verifier independently validates the minimum sufficient evidence. A business must not unilaterally disclose protected counterparty fields; some disclosures need both parties' consent.

Revocation stops future network-authorized access but cannot retract information already viewed, downloaded, or copied. Record grants and disclosures for audit without publicly exposing sensitive financing/audit activity.

9. Ledger and external-chain strategy

The network's own distributed ledger is the authoritative triple-entry system. An external chain is optional and is not a substitute for protocol security, key management, identity proofing, encrypted storage, or governance.

Possible later use: periodically anchor a Merkle root or opaque batch commitment to make retrospective rewriting more detectable across governance domains. Do not publish plaintext, guessable document hashes, or unnecessary business metadata. Define exactly what the anchor proves and does not prove, how outages/forks are handled, and how migration works.

Evaluate candidates for validator independence, finality, privacy leakage, governance/upgrade controls, availability, cost at expected volume, jurisdictional risk, and exit capability. This blueprint does not endorse a specific external chain.

10. Identity, governance, and trust boundaries

- Separate business identity proofing, node admission, and transaction-signing authority.
- Support multiple authorized users/devices, delegated roles, and auditable key rotation/revocation.
- Define validator admission/removal, upgrade thresholds, emergency response, and version compatibility.
- Prevent any mother node or institution from being a hidden single point of control for discovery, identity, or software updates.
- Publish verifiable software releases and network configuration; define secure bootstrap and recovery.
- Document metadata visible to relays/validators: routing, size, timing, and frequency. Encryption alone does not hide traffic patterns.

11. Interoperability and integrations

Use a canonical document model with versioned profiles/codecs for GST e-invoice and e-way bill, ERP connectors, and future ONDC/Beckn, OCEN, and EDIFACT work. External rails are explicit disclosure boundaries: data sent to them is visible to those providers under applicable terms and law.

Implemented: Zoho Books adapter prototype

The POC has a Zoho Books-oriented HTTP client and invoice-sync adapter. The flow fetches an invoice, maps it to the canonical trade-document schema, creates an encrypted envelope, builds and submits a bilateral signed ledger proposal, then sends the resulting event metadata/status back to the ERP client. The flow is tested against a fake Zoho HTTP service; it is not yet verified against a live Zoho organization or production API contract. Configuration currently accepts a base URL, optional access token and organization ID, and request timeout. OAuth authorization/refresh, secret storage/rotation, real-account compatibility, retry/reconciliation, and operational audit remain future work. Other ERP and external-rail codecs are not implemented.

Implemented: sovereign AI gateway foundation

CompanyAgentGateway models each company's agent as acting through its own company-controlled gateway. It can sign versioned intents and check expiry, allowed actions, document IDs, and recipients. This establishes a local policy boundary; it does not connect independently hosted company gateways. There is no inter-company HTTP endpoint, queue/event bus, webhook callback, durable replay/idempotency store, or trusted cross-company key-discovery service yet. The signed-intent exchange must be treated as a protocol foundation, not live agent-to-agent networking.

For production, keep ERP adapters and AI gateways in the participant's controlled environment where practical. Cloud connectors should use narrowly scoped credentials and minimum required data. Bind externally submitted results to a source document revision and record them as signed events. Add asynchronous delivery, acknowledgements, retries, idempotency, dead-letter handling, and auditable authorization before relying on cross-company events operationally.

12. Initial component boundaries

canonical/	implemented: canonical JSON, trade-document schema, validation; semantic TEA posting model in progress
crypto/	implemented POC: signing, verification, envelope encryption; production key custody pending
identity/	pending: business credentials, public-key discovery, delegation, revocation
ledger/	implemented POC: bilateral proposals plus signed provisional/accept/dispute TEA events in an in-memory append-only
network/	implemented POC: single-process HTTP sandbox with TEA event/status routes; peer discovery, relay, persistence,
storage/	pending: durable encrypted payloads, replication, retention, backup/recovery
consent/	pending: scoped grants, disclosure packaging, verifier/audit interface
connectors/	implemented prototype: Zoho Books invoice mapping to balanced postings and provisional TEA submission; live OA
agent-gateway/	implemented foundation: signed intents and local policy checks; transport/event delivery pending
node-runtime/	pending: node capabilities, production configuration, monitoring, safe upgrades

Canonicalization and validation should be deterministic and independently tested. Network-facing code validates size, version, authorization, signatures/proofs, replay, and state transitions before acceptance. Secrets and plaintext must not cross module boundaries without need.

13. Build sequence and exit criteria

Phase 0: protocol and threat model (partially implemented)

Deliver actor/data-flow diagrams, privacy/metadata matrix, node capability model, trust assumptions, identity/key lifecycle, event specification, consensus/finality choice, and legal retention review. Exit when the team can state who sees each field, who validates each rule, when events are final, and how key loss, validator outage, disputes, and corrections work.

Phase 1: two-party private exchange (core primitives implemented; relay flow pending)

Deliver canonical document, local keys, signed/encrypted message, untrusted relay, recipient verification, idempotent retries, encrypted local storage. Exit when a relay cannot read or silently alter a message and both parties independently verify the same revision.

Phase 2: distributed triple-entry ledger (semantic event lifecycle prototype implemented; persistence and distributed operation pending)

Deliver event schema/accounting rules, multi-operator validator prototype, countersignature flow, append-only corrections/disputes, replicated verification, finality and recovery tests. Exit when independent nodes converge and reject tampered, replayed, unauthorized, or conflicting events under documented fault assumptions.

Phase 3: node classes and governance (pending)

Deliver node profiles, capability enforcement, signed releases/configuration, operator onboarding, and monitoring without sensitive payload logging. Exit when no single operator is required for ordinary exchange or verification and governance actions are independently auditable.

Phase 4: third-party verification (pending; agent gateway is not a verifier-grant system)

Deliver signed scoped grants, verifier identity, authenticity-only and selected-field flows, consent audit, expiry/revocation checks, counterparty data protections. Exit when a verifier can prove only authorized claims and cannot retrieve unrelated records.

Phase 5: targeted ZK and external anchoring (pending)

Only proceed for a concrete use case. Deliver audited proof statements/circuits, test vectors, benchmarks, upgrade policy, and/or a privacy-reviewed anchoring prototype. Exit when it adds measurable capability over signatures/commitments/selective disclosure and passes independent review.

Phase 6: pilot and production hardening (pending)

Deliver independent security/cryptographic review, penetration test, backup/restore and key-compromise drills, incident response, load/finality tests, legal/compliance review, operator SLAs, support, and migration plans.

14. Key risks and mitigations

Risk	Mitigation direction
Visible accounting data reveals commercial relationships/amounts	Field visibility policy, private channels, commitments/proofs, metadata minimization
Lost or compromised business keys	Local/hardware-backed custody where practical, delegation, rotation/revocation, tested recovery and explicit trust trade-offs
Incorrect canonicalization or proof circuit	Versioned deterministic specs, test vectors, independent review, controlled upgrades
Permanent evidence conflicts with retention/erasure duties	Separate encrypted payloads, define lawful retention and key-destruction consequences before launch
Mother services centralize control	Independent operators, signed public config, transparent governance and exit paths
Validator outage or collusion	Explicit fault threshold, independent operators, monitoring, recovery/checkpoint procedures
Third-party grant exceeds scope	Audience-bound grants, least privilege, expiry, independent verification and audit
External-chain metadata/dependency risks	Privacy-review batch commitments; optional, replaceable anchoring
Sandbox is mistaken for a production network	Keep it isolated; move all company key operations to participant gateways, add durable storage, rate limiting, production identity and authorization, operational monitoring, and independent security review before deployment
ERP or agent integration is assumed to be production-ready	Validate live provider APIs and OAuth lifecycle; add trusted identity, transport, replay protection, retries, and operational monitoring before enabling real workflows

15. Decisions required before architecture is fixed

1. Is the initial ledger permissioned among admitted institutions, open to qualifying operators, or hybrid?
2. Which actors validate events, and what collusion/outage threshold must finality tolerate?
3. Which fields are visible to counterparties, validators, other participants, and third-party verifiers?
4. Is GSTIN visible on-ledger, pseudonymous, or selectively disclosed?
5. What key recovery model is acceptable, and which parties are trusted in recovery?
6. Which event kinds may be provisionally submitted by one party, and which require counterparty acceptance before confirmation?
7. What retention, deletion, backup, and legal-hold obligations apply to payloads and ledger evidence?
8. What is the first third-party verification use case and exact claim to prove?
9. What measurable requirement justifies ZK or an external chain over signatures and commitments?
10. What availability, throughput, offline, and cost targets define pilot success?

16. Working recommendation

Build the exchange and triple-entry ledger as one protocol with separate privacy boundaries: signed accounting events and commitments form shared history; encrypted documents are delivered/replicated for authorized participants; scoped grants let third parties verify selected claims. Begin with signatures, encryption, and commitments. Introduce ZK proofs or external-chain anchors only when a concrete workflow demonstrates value. Choose consensus and node governance before selecting a blockchain product.

The current POC demonstrates canonical document validation, signatures and encrypted envelopes, legacy bilateral ledger-proposal verification, balanced semantic posting sets for INR invoices/credit notes, and a signed provisional-to-confirmed/disputed TEA lifecycle in an in-memory HTTP sandbox. The Zoho flow submits a seller-signed provisional event, and the buyer can verify the salted commitment and countersign; the provider test uses a fake Zoho HTTP service. It does not yet demonstrate durable storage, independently operated nodes, a blind untrusted relay, distributed consensus/finality, scoped third-party verifier grants, production Zoho OAuth, or transport between company gateways.

The next architecture steps are to persist the event log, add idempotent ERP retries and buyer-side reconciliation, define production business identity and scoped authorization, separate the relay from participant-controlled signing/encryption, and specify cross-company intent trust/replay semantics. The HTTP API now accepts public identities and pre-encrypted envelopes, but the sandbox remains an in-memory prototype; use it only behind controlled access and TLS. Production deployment still requires governance, operational, legal, and independent security review.